Doc No: P1438R0
Date: 2019-01-19
Author: Bill Seymour
Reply to: stdbill.h@pobox.com
Audience: SG6 (Numerics)

A Rational Number Library for C++

Bill Seymour
2019-01-19

Introduction:

This paper proposes a rational number library for a C++ Technical Specification (TS).

Because almost every operation on rational numbers requires multiplication behind the scenes, numerators and denominators can become quite large in a hurry, easily overflowing any fundamental integer type; so this paper requires that the numerator and denominator be stored internally in an unlimited-precision integer such as the one suggested by Pete Becker in N4038.

Revision history:

- N3363 – post-Kona (original)
- N3414 – pre-Portland
- N3489 – post-Portland
- N3611 – pre-Bristol
- DnnnnR0 – this paper

It just occurred to me that this never made it into a mailing. This paper contains changes that SG6 suggested at the last Kona meeting. It's the latest and greatest. I don't anticipate needing committee time in the 2019 Kona meeting.

Many thanks to Daniel Krügler for several excellent suggestions to get from N3363 to N3414.

Thanks also to Lawrence Crowl for his excellent suggestions for this revision.

As suggested in Bristol, normalization is now required only when the numerator and denominator become separately visible as a result of calling the `numer()` and `denom()` observers or the `ostream` insertion operator, or when the user explicitly requests it by calling the `normalize()` member function. The implementation is permitted to reduce the fraction to lowest terms at other times for efficiency, but that's a quality-of-implementation issue.

The rounding mode argument to the `nearest` function now has the `enum class rounding` type in Lawrence Crowl's P0105R1.

The `seminumeric` namespace is a placeholder for whatever namespace will appear in the TS.

Questions:

1. Do we want conversion from fixed- and/or floating-point decimals? Fixed-point binary? I think all of them; but it's not clear yet what will come out of SG-6 in this regard.

2. Do we want rational literals? I think not; and I echo the rationale in Pete Becker's Open issue 7 in N4038 (they wouldn't be `constexpr`).

[rational.math] Rational math

The header `<rational>` defines the `rational` class and several free functions and operators for doing arithmetic with rational numbers.

Header `<rational>` synopsis

```
namespace std {
namespace experimental {
namespace seminumeric {

class rational;

rational operator+(const rational& val);
rational operator-(const rational& val);

rational operator+(const rational& lhs, const rational& rhs);
rational operator-(const rational& lhs, const rational& rhs);
rational operator*(const rational& lhs, const rational& rhs);
rational operator/(const rational& lhs, const rational& rhs);

rational operator+(const rational& lhs, const integer& rhs);
rational operator-(const rational& lhs, const integer& rhs);
rational operator*(const rational& lhs, const integer& rhs);
rational operator/(const rational& lhs, const integer& rhs);

rational operator+(const integer& lhs, const rational& rhs);
rational operator-(const integer& lhs, const rational& rhs);
rational operator*(const integer& lhs, const rational& rhs);
rational operator/(const integer& lhs, const rational& rhs);

bool operator==(const rational& lhs, const rational& rhs);
bool operator!=(const rational& lhs, const rational& rhs);
bool operator< (const rational& lhs, const rational& rhs);
bool operator> (const rational& lhs, const rational& rhs);
bool operator<=(const rational& lhs, const rational& rhs);
bool operator>=(const rational& lhs, const rational& rhs);

bool operator==(const rational& lhs, const integer& rhs);
bool operator!=(const rational& lhs, const integer& rhs);
bool operator< (const rational& lhs, const integer& rhs);
bool operator> (const rational& lhs, const integer& rhs);
bool operator<=(const rational& lhs, const integer& rhs);
bool operator>=(const rational& lhs, const integer& rhs);

bool operator==(const integer& lhs, const rational& rhs);
bool operator!=(const integer& lhs, const rational& rhs);
bool operator< (const integer& lhs, const rational& rhs);
bool operator> (const integer& lhs, const rational& rhs);
bool operator<=(const integer& lhs, const rational& rhs);
bool operator>=(const integer& lhs, const rational& rhs);
```

```
void swap(rational& lhs, rational& rhs);

integer floor(const rational& val);
integer ceil (const rational& val);
integer trunc(const rational& val);
integer nearest(const rational& val, rounding mode = rounding::tie_to_even);
integer round(const rational& val);

rational abs(const rational& val);

rational reciprocal(const rational& val);

rational pow(const rational& val, const integer& exp);

template<class intT>
  rational modf(const rational& val, intT* intptr);

rational modf(const rational& val);

template<class charT, class traits>
  std::basic_ostream<charT,traits>&
    showden1(std::basic_ostream<charT,traits>& stream);

template<class charT, class traits>
  std::basic_ostream<charT,traits>&
    noshowden1(std::basic_ostream<charT,traits>& stream);

template<class charT, class traits>
  std::basic_ostream<charT,traits>&
    divalign(std::basic_ostream<charT,traits>& stream);

template<class charT, class traits>
  std::basic_ostream<charT,traits>&
    nodivalign(std::basic_ostream<charT,traits>& stream);

template<class charT>
  implementation-detail setdiv(charT divsign);

template<class charT, class traits>
  std::basic_istream<charT,traits>&
    operator>>(std::basic_istream<charT,traits>& lhs, rational& rhs);

template<class charT, class traits>
  std::basic_ostream<charT,traits>&
    operator<<(std::basic_ostream<charT,traits>& lhs, const rational& rhs);

} // namespace seminumeric
} // namespace experimental
} // namespace std
```

[rational.class] Class `rational`

```
class rational {
    mutable integer this_num, this_den; // exposition only

public:
    rational() noexcept;

    rational(const rational& rat);
    rational(rational&& rat);

    explicit rational(float num);
    explicit rational(double num);
```

```
    explicit rational(long double num);

    explicit rational(const integer& num);
    explicit rational(integer&& num) noexcept;

    rational(const integer& num, const integer& den);
    rational(integer&& num, integer&& den) noexcept;

    ~rational();

    rational& operator=(const rational& rat);
    rational& operator=(rational&& rat) noexcept;

    rational& operator=(const integer& num);
    rational& operator=(integer&& num);

    rational& assign(const integer& num, const integer& den);
    rational& assign(integer&& num, integer&& den) noexcept;

    void swap(rational& rhs) noexcept;

    void normalize() const;

    integer numer() const;
    integer denom() const;

    explicit operator bool() const noexcept;

    rational& negate() noexcept;
    rational& invert() noexcept;

    rational& operator++();
    rational& operator--();

    rational operator++(int);
    rational operator--(int);

    rational& operator+=(const integer& rhs);
    rational& operator-=(const integer& rhs);
    rational& operator*=(const integer& rhs);
    rational& operator/=(const integer& rhs);

    rational& operator+=(const rational& rhs);
    rational& operator-=(const rational& rhs);
    rational& operator*=(const rational& rhs);
    rational& operator/=(const rational& rhs);
};
```

The numerator and denominator shall be stored internally in a std::experimental::seminumeric::integer.

The internal representation shall be reduced to lowest terms (the numerator and denominator shall have no common factor other than 1 and the denominator shall be greater than zero):

- as a result of calling the `normalize()` member function, and
- when the numerator and/or denominator become separately visible (as a result of calling the `numer()` and `denom()` observers or the `ostream` insertion operator).

The implementation may also normalize the representation at other times for the sake of efficiency.

[rational.members] `rational` member functions:

```
rational() noexcept;
```

Effects: Constructs a `rational` with a value of zero.

```
rational(const rational& rat);
rational(rational&& rat);
```

Effects: Constructs a `rational` with a value of `rat`.

```
explicit rational(const integer& num);
explicit rational(integer&& num);
```

Effects: Constructs a `rational` with a value of `num`.

```
explicit rational(float val);
explicit rational(double val);
explicit rational(long double val);
```

Effects: constructs a `rational` with a value equal to `val`.

```
rational(const integer& num, const integer& den);
rational(integer&& num, integer&& den) noexcept;
```

Requires: `den != 0`

Effects: Constructs a `rational` given the specified numerator and denominator.

```
~rational();
```

Effects: Destructs `*this`.

```
rational& operator=(const rational& rhs);
rational& operator=(rational&& rhs) noexcept;
```

Effects: Assigns `rhs` to `*this`.

Returns: `*this`.

```
rational& operator=(const integer& num);
rational& operator=(integer&& num);
```

Effects: Assigns `num` to `*this`.

Returns: `*this`.

```
rational& assign(const integer& num, const integer& den);
rational& assign(integer&& num, integer&& den) noexcept;
```

Requires: `den != 0`

Effects: Assigns the specified numerator and denominator to `*this`.

Returns: `*this`.

```
void swap(rational& rhs) noexcept;
```

Effects: Swaps `*this` and `rhs`.

`void normalize() const;`[footnote]

Effects: reduces the fraction to lowest terms.

Postcondition: `numer()` and `denom()` have no common factors other than 1, and `denom()` is greater than zero.

[footnote]This is a conceptually `const` member function because it's used by the `numer()` and `denom()` observers and the `ostream` insertion operator. Note that the observable value of `*this` doesn't change, which is consistent with the meaning of `const`.

```
integer numer() const;
```

Effects: reduces the fraction to lowest terms as if by calling `normalize()`.

Returns: the numerator by value.

```
integer denom() const;
```

Effects: reduces the fraction to lowest terms as if by calling `normalize()`.

Returns: the denominator by value.

```
explicit operator bool() const noexcept;
```

Returns: as if `numer() != 0`.

```
rational& negate() noexcept;
```

Effects: changes the sign of the numerator.

Returns: `*this`.

```
rational& invert() noexept;
```

Effects: swaps the numerator and denominator, changing their signs if necessary to keep the denominator positive.

Requires: the numerator is non-zero.

Returns: `*this`.

[rational.member.ops] `rational` member operators:

```
rational& operator++();
```

Effects: adds 1 to `*this` and stores the result in `*this`.

Returns: `*this`.

```
rational& operator--();
```

Effects: subtracts 1 from `*this` and stores the result in `*this`.

Returns: `*this`.

```
rational operator++(int);
```

Effects: adds 1 to `*this` and stores the result in `*this`.

Returns: the value of `*this` before the addition.

`rational operator--(int);`

Effects: subtracts 1 from `*this` and stores the result in `*this`.

Returns: the value of `*this` before the subtraction.

`rational& operator+=(const integer& rhs);`

Effects: adds the integer value `rhs` to `*this` and stores the result in `*this`.

Returns: `*this`.

`rational& operator-=(const integer& rhs);`

Effects: subtracts the integer value `rhs` from `*this` and stores the result in `*this`.

Returns: `*this`.

`rational& operator*=(const integer& rhs);`

Effects: multiplies `*this` by the integer value `rhs` and stores the result in `*this`.

Returns: `*this`.

`rational& operator/=(const integer& rhs);`

Effects: divides `*this` by the integer value `rhs` and stores the result in `*this`.

Requires: `rhs != 0`.

Returns: `*this`.

`rational& operator+=(const rational& rhs);`

Effects: adds the rational value `rhs` to `*this` and stores the result in `*this`.

Returns: `*this`.

`rational& operator-=(const rational& rhs);`

Effects: subtracts the rational value `rhs` from `*this` and stores the result in `*this`.

Returns: `*this`.

`rational& operator*=(const rational& rhs);`

Effects: multiplies `*this` by the rational value `rhs` and stores the result in `*this`.

Returns: `*this`.

`rational& operator/=(const rational& rhs);`

Effects: divides `*this` by the rational value `rhs` and stores the result in `*this`.

Requires: `rhs != 0`.

Returns: `*this`.

[rational.ops] `rational` non-member operations:

```
rational operator+(const rational& val);
```

Returns: `rational(val)`.

```
rational operator-(const rational& val);
```

Returns: `rational(val).negate()`.

```
rational operator+(const rational& lhs, const rational& rhs);
```

Returns: `rational(lhs) += rhs`.

```
rational operator-(const rational& lhs, const rational& rhs);
```

Returns: `rational(lhs) -= rhs`.

```
rational operator*(const rational& lhs, const rational& rhs);
```

Returns: `rational(lhs) *= rhs`.

```
rational operator/(const rational& lhs, const rational& rhs);
```

Requires: `rhs != 0`.

Returns: `rational(lhs) /= rhs`.

```
rational operator+(const rational& lhs, const integer& rhs);
```

Returns: `rational(lhs) += rhs`.

```
rational operator-(const rational& lhs, const integer& rhs);
```

Returns: `rational(lhs) -= rhs`.

```
rational operator*(const rational& lhs, const integer& rhs);
```

Returns: `rational(lhs) *= rhs`.

```
rational operator/(const rational& lhs, const integer& rhs);
```

Requires: `rhs != 0`.

Returns: `rational(lhs) /= rhs`.

```
rational operator+(const integer& lhs, const rational& rhs);
```

Returns: `rational(rhs) += lhs`.

```
rational operator-(const integer& lhs, const rational& rhs);
```

Returns: `rational(rhs).negate() += lhs`.

```
rational operator*(const integer& lhs, const rational& rhs);
```

Returns: `rational(rhs) *= lhs`.

```
rational operator/(const integer& lhs, const rational& rhs);
```

Requires: `rhs != 0`.

Returns: `rational(rhs).invert() *= lhs`.

```
bool operator==(const rational& lhs, const rational& rhs);
```

Returns: as if `lhs.numer() == rhs.numer() && lhs.denom() == rhs.denom()`.

```
bool operator!=(const rational& lhs, const rational& rhs);
```

Returns: `!(lhs == rhs)`.

```
bool operator<(const rational& lhs, const rational& rhs);
```

Returns: as if `lhs.numer() * rhs.denom() < rhs.numer() * lhs.denom()`.

```
bool operator>(const rational& lhs, const rational& rhs);
```

Returns: `rhs < lhs`.

```
bool operator<=(const rational& lhs, const rational& rhs);
```

Returns: `lhs < rhs || lhs == rhs`.

```
bool operator>=(const rational& lhs, const rational& rhs);
```

Returns: `lhs > rhs || lhs == rhs`.

```
bool operator==(const rational& lhs, const integer& rhs);
```

Returns: as if `lhs.numer() == rhs && lhs.denom() == 1`.

```
bool operator!=(const rational& lhs, const integer& rhs);
```

Returns: `!(lhs == rhs)`.

```
bool operator<(const rational& lhs, const integer& rhs);
```

Returns: as if `lhs.numer() < rhs * lhs.denom()`.

```
bool operator>(const rational& lhs, const integer& rhs);
```

Returns: as if `lhs.numer() > rhs * lhs.denom()`.

```
bool operator<=(const rational& lhs, const integer& rhs);
```

Returns: `lhs < rhs || lhs == rhs`.

```
bool operator>=(const rational& lhs, const integer& rhs);
```

Returns: `lhs > rhs || lhs == rhs`.

```
bool operator==(const integer& lhs, const rational& rhs);
```

Returns: `rhs == lhs`.

```
bool operator!=(const integer& lhs, const rational& rhs);
```

Returns: `rhs != lhs`.

```
bool operator<(const integer& lhs, const rational& rhs);
```

Returns: `rhs > lhs`.

```
bool operator>(const integer& lhs, const rational& rhs);
```

Returns: `rhs < lhs`.

```
bool operator<=(const integer& lhs, const rational& rhs);
```

Returns: `rhs >= lhs`.

```
bool operator>=(const integer& lhs, const rational& rhs);
```

Returns: `rhs <= lhs`.

[rational.conv] `rational` numeric conversions:

```
integer floor(const rational& val);
```

Returns: the most positive integer not greater than `val`.

```
integer ceil(const rational& val);
```

Returns: the most negative integer not less than `val`.

```
integer trunc(const rational& val);
```

Returns: the integer part of `val` truncated toward zero.

```
integer nearest(const rational& val, rounding mode = rounding::tie_to_even);
```

Returns: the integer nearest in value to `val`. In the case of a tie (if `val.denom()` would be 2), the returned value shall be rounded as specified by `mode`.

```
integer round(const rational& val);
```

Returns: `nearest(val, rounding::all_away_zero)`

[rational.swap] `rational` swap:

```
void swap(rational& lhs, rational& rhs) noexcept;
```

Effects: `lhs.swap(rhs)`.

[rational.manip] I/O manipulators that apply to `rational` objects:

```
template<class charT, class traits>
  std::basic_ostream<charT,traits>&
    showden1(std::basic_ostream<charT,traits>& stream);

template<class charT, class traits>
  std::basic_ostream<charT,traits>&
    noshowden1(std::basic_ostream<charT,traits>& stream);
```

These output manipulators control whether a denominator equal to 1 is written. If `noshowden1` is in effect and the `rational`'s denominator equals 1, the output shall be just the numerator written as an ordinary `integer`. The default is `noshowden1`.

```
template<class charT, class traits>
  std::basic_ostream<charT,traits>&
    divalign(std::basic_ostream<charT,traits>& stream);

template<class charT, class traits>
  std::basic_ostream<charT,traits>&
    nodivalign(std::basic_ostream<charT,traits>& stream);
```

These output manipulators specify whether the stream's width and `adjustfield` flags affect the numerator only (`divalign`) or the whole fraction (`nodivalign`). The default is `nodivalign`. [*Note:* `divalign` is intended for printing fractions in columns by aligning the division signs. — *end note*]

[Example —

```
rational r1(5);
rational r2(24, 17);
cout << r1 << '\n'
     << r2 << '\n'
     << showden1
     << setfill('*')
     << setw(6) << r1 << '\n'
     << setw(6) << r2 << '\n'
     << divalign
     << setw(6) << r1 << '\n'
     << setw(6) << r2 << '\n'
     << left << setfill(' ') << setw(6) << r2
     << " (You might want to avoid left with divalign.)\n";
```

yields the output:

```
5
24/17
***5/1
*24/17
*****5/1
****24/17
24    /17 (You might want to avoid left with divalign.)
```

— end example]

```
template<class charT>
  implementation-detail setdiv(charT divsign);
```

This I/O manipulator causes `divsign` to be used for the division sign. The default is the solidus (*stream*`.widen('/')`).

[rational.io] I/O operators that apply to `rational` objects:

```
template<class charT, class traits>
  std::basic_istream<charT,traits>&
    operator>>(std::basic_istream<charT,traits>& lhs, rational& rhs);
```

The extraction operator reads an integer value which it takes to be a numerator, and if that is immediately followed by a division sign, reads another integer value which it takes to be a denominator. If no division sign immediately follows the numerator, the denominator shall be assumed to be 1.

If the operator reads a value of zero when expecting a denominator, it shall set the stream's `failbit`. If an `ios_base::failure` exception is thrown, or if the stream is not `good()` when the operator returns, `rhs` shall be untouched.

The stream's `locale` and all its format flags except `skipws` shall apply separately to the numerator and the denominator. `skipws` shall apply to the numerator only.[footnote]

[footnote]Thus, if a denominator is present, the division sign must immediately follow the numerator, and the denominator must immediately follow the division sign, without any intervening characters including whitespace.

```
template<class charT, class traits>
  std::basic_ostream<charT,traits>&
    operator<<(std::basic_ostream<charT,traits>& lhs, const rational& rhs);
```

The insertion operator shall write a `rational` to the specified output stream; and the numerator and denominator shall be written in whatever format is appropriate given the stream's flags and `locale`. The `showpos` and `showbase` flags shall affect the numerator only. The output shall be in lowest terms as if `rhs.normalize()` had been called just prior to doing the output.

[rational.over] Additional overloads:

```
rational abs(const rational& val);
```

Returns: `rational(val)` if `val` >= 0; otherwise `-val`.

```
rational reciprocal(const rational& val);
```

Returns: `rational(val).invert()`.

```
rational pow(const rational& val, const integer& exp);
```

Requires: `val != 0 || exp >= 0`.

Returns: `rational(1)` if `exp` == 0; otherwise `val` raised to the `exp` power.

Remarks: the exponent must be an integer since raising to a non-integer power yields an irrational value in general.

```
template<class intT>
  rational modf(const rational& val, intT* intptr);
```

Effects: if `intptr` is not a null pointer, `*intptr` shall be assigned `trunc(val)`.

Returns: `val - trunc(val)`.

```
rational modf(const rational& val);
```

Returns: `val - trunc(val)`.

All suggestions and corrections will be welcome; all flames will be amusing.

Bill Seymour, stdbill.h@pobox.com.